LABORATORY PRACTICAL REPORT & SUBMISSION

Design, Development & Deployment of a ReactJS Web Application Demonstrating Client-Side Validation and Interactive UX

Course / Subject:	Web Technology (WT) Lab	**Academic Year:**	3rd Year Engineering (2025-2026)
Project Name:	EventSphere 2026 Summit Portal	**Technology Stack:**	ReactJS 19, Vite 8, Modern CSS3
Live Web URL:	https://teamstreetup.github.io/eventsphere-react-validation/	**Deployment Host:**	GitHub Pages (Continuous Deployment)
GitHub Repository:	https://github.com/teamstreetup/eventsphere-react-validation	**Automated Tests:**	**42 / 42 Tests Passed (100% Pass)**

1. Problem Statement & Assignment Objectives

The objective of this assignment is to design, develop, test, and deploy an individual web application using **ReactJS** that accepts structured user input, performs thorough **client-side validation**, displays intuitive and accessible error feedback, provides positive confirmation messaging upon valid entry, and offers an engaging, interactive user experience. The application must model a real-world scenario and be published live on the web.

2. Selected Real-World Domain: EventSphere 2026 Summit Portal

To satisfy the real-world requirement, **EventSphere 2026** was conceived as a high-performance developer conference and hands-on workshop registration portal. It captures diverse input types across four progressive stages:

- **Step 1: Personal Identification:** Full legal name, verified email, 10-digit mobile number, date of birth (enforcing an age constraint >= 18), and gender identity.
- **Step 2: Academic & Professional Profile:** College/organization affiliation, professional role, technical track specialization, optional portfolio/GitHub URL, and a statement of purpose with real-time character boundary monitoring (30 to 300 characters).
- **Step 3: Access Pass & Workshop Tracks:** Pass tier selection (Student, Developer, VIP), hands-on workshop selection (minimum 1, maximum 3), apparel sizing, dietary preferences, and dynamic coupon discount engine.
- **Step 4: Account Security & Final Confirmation:** 5-point password strength verification, character-by-character password confirmation, terms of service acceptance, and registration review summary.

3. Form Validation Architecture & Rule Matrix

The application utilizes pure JavaScript validator functions decoupled from the view layer (`src/utils/validation.js`), enabling instantaneous UI feedback during keystroke (`onChange`) and element departure (`onBlur`) events without premature user fatigue.

Input Field	Validation Technique & Criteria	Error Trigger & Message	Valid State Indicator
Full Name	Regex: ^[A-Za-z]+([A-Za-z'-]+)*$ Length: 3 to 50 chars	• < 3 characters • Contains digits or symbols	Green checkmark + border
Email Address	RFC 5322 regex pattern + Domain typo heuristic (@gmai.com, @yaho.com)	• Missing '@' or top-level domain • Suggests typo correction	Verified domain pattern
Mobile Number	Regex: ^[6-9]\d{9}$ Exact 10 digits starting with 6,7,8,9	• Length != 10 digits • Invalid prefix or non-numeric	Valid telecom format
Date of Birth	Temporal math: Age = (Today - DOB) >= 18 Rejects future & unrealistic dates	• Age < 18 years old • Date in the future	Age threshold confirmed
Institution	Required string; trimmed length >= 3 chars	• Empty or whitespace only • Length < 3 chars	Affiliation recorded
Portfolio URL	Optional; if entered: requires http:// or https:// with valid domain syntax	• Malformed URL syntax • Missing protocol header	Clean link format
Bio / SOP	Real-time string length bounds: Min 30 chars, Max 300 chars	• Length < 30 chars • Length > 300 chars	Counter color-coded green
Workshops	Checkbox array cardinality: 1 <= selected <= 3	• 0 workshops selected • > 3 workshops chosen	Valid schedule selection
Password	5-factor criteria: 8+ chars, uppercase, lowercase, number, special character	• Fewer than 8 chars • Strength score < 4 out of 5	Dynamic green strength meter
Confirm Pass	Strict equality check: confirmPassword === password	• Password mismatch detected	Exact character match
Terms Check	Boolean truth evaluation: agreeTerms === true	• Checkbox unchecked on submit	Legally acknowledged

4. Interactive Features & State Management Architecture

EventSphere 2026 goes beyond static forms by offering several sophisticated reactive features powered by React hooks:

- **Progressive Multi-Step Disclosure with Validation Gating:** The user progresses through 4 logically segmented views. Progression to the subsequent step is strictly gated; attempting to proceed with invalid inputs immediately focuses the affected fields, marks them as touched, and displays animated error banners.
- **Reactive Live Price Engine:** Dynamic pricing is computed via React's useMemo hook. The pricing formula computes: Total = PassPrice + ExtraWorkshopFees - Discount. Workshop fees are dynamically waived when the VIP Pass is chosen, demonstrating dynamic business rules.
- **Instant Coupon Code Verification:** Users can test coupon codes (STUDENT50: 50% discount, EARLYBIRD: 25% discount, DEVCON: flat Rs. 300 off) with instantaneous subtotal adjustments or shake alerts for invalid codes.
- **Password Strength Inspector:** Evaluates passwords across 5 criteria in real-time, providing immediate visual confirmation via an animated progress bar that transitions from Red (Weak) to Orange, Amber, and Green (Strong).
- **Local Persistence & Directory:** Completed registrations persist inside the browser's localStorage, populating a searchable and filterable Attendee Directory with real-time revenue analytics and JSON export capabilities.
- **Digital Pass Generation & Confetti:** On valid form submission, a celebratory particle burst is triggered using canvas-confetti, and an interactive Digital Event Pass is rendered complete with a simulated check-in QR code, badge ID, and print stylesheet.
- **Examiner Demonstration Sandbox:** The interface includes 'Fill Valid Data' and 'Test Error States' buttons, allowing evaluators to immediately verify both success and failure flows with a single click.

5. Verification & Test Execution Results

To rigorously verify client-side validation accuracy across edge cases, an automated test runner (`test-validation.js`) was executed using Node.js. All 42 unit test assertions passed with zero defects:

Test Module	Test Scenarios Evaluated	Sample Test Inputs	Outcome
Full Name	Min length, empty string, symbols, numbers, valid hyphens	'Aarav Sharma' (Pass), 'Jo' (Fail), 'Aarav123' (Fail)	**5 / 5 Passed**
Email	RFC compliance, subdomain support, common typo heuristic	'student@college.edu' (Pass), 'user@gmai.com' (Typo flagged)	**6 / 6 Passed**
Phone	Length boundary (9, 10, 11), numeric filter, prefix check	'9876543210' (Pass), '1234567890' (Fail), '98765' (Fail)	**6 / 6 Passed**
Age / DOB	Adult age >= 18, minor boundary (< 18), future dates	'2000-01-15' (Pass), '2020-05-10' (Fail), '2030-01-01' (Fail)	**4 / 4 Passed**
Institution	String length check, whitespace trimming, empty rejection	'Maharashtra Institute of Tech' (Pass), 'AB' (Fail)	**3 / 3 Passed**
Bio / SOP	Underflow boundary (< 30 chars), valid length (30-300)	'Passionate computer engineering student...' (Pass), 'Short' (Fail)	**3 / 3 Passed**
Workshops	Cardinality limits: empty array, 1 track, 3 tracks, 4 tracks	['ws1'] (Pass), [] (Fail), ['ws1'..'ws4'] (Fail)	**4 / 4 Passed**
Password	5-factor complexity scoring, length >= 8, regex checks	'SecurePass@2026' (Score 5/5), '123' (Score 1/5)	**5 / 5 Passed**
Confirm Pass	Identical match versus intentional typo mismatch	'Match@123' == 'Match@123' (Pass), 'Mis' != 'Match' (Fail)	**2 / 2 Passed**
Full Form	End-to-end integration test of valid attendee vs invalid state	validateAll(SAMPLE_VALID_USER) → isValid: true	**4 / 4 Passed**

6. Project Structure & Code Architecture

The project follows a clean separation of concerns, ensuring maximum readability, testability, and maintainability:

Directory / File	Responsibility & Implementation Details
`src/App.jsx`	Main orchestrator; manages multi-step state, active navigation tabs, dynamic pricing memoization, and submission handlers.
`src/utils/validation.js`	Pure validation library containing regex patterns, age calculations, password scoring algorithms, and step/all validation gates.
`src/utils/constants.js`	Application data models: pass tiers, workshop tracks, coupon dictionaries, and preset evaluation personas.
`src/components/FormField.jsx`	Reusable form field wrapper managing labels, asterisks, aria alerts, inline error icons, and success states.
`src/components/StepOnePersonal.jsx`	Step 1 component: Full Name, RFC Email, 10-Digit Mobile, Age 18+ Date of Birth, and Gender.
`src/components/StepTwoProfile.jsx`	Step 2 component: College/Company, Track selector, Portfolio URL validation, and 30-300 char counter Bio.
`src/components/StepThreeTickets.jsx`	Step 3 component: Pass cards, workshop checkbox group (min 1, max 3), and reactive coupon price engine.
`src/components/StepFourSecurity.jsx`	Step 4 component: 5-point password strength bar, show/hide toggle, password confirmation, and terms checkbox.
`src/components/RegistrationSummaryModal.jsx`	Confirmation dialog: triggers confetti animation, renders digital event badge with QR code, and print styling.
`src/components/AttendeesList.jsx`	Tab 2 view: LocalStorage attendee table with real-time text search, pass-tier filtering, and JSON export.
`src/components/ValidationGuide.jsx`	Tab 3 view: In-app technical reference guide displaying all regex formulas and validation criteria.
`test-validation.js`	Automated unit testing script verifying 42 assertions using Node.js.

7. Viva Voce Technical Questions & Model Answers

Q1: Why is client-side validation crucial, and can it replace server-side validation?

Answer: Client-side validation delivers instant feedback to the user, catches syntax/boundary errors without network latency, and conserves server resources. However, it *cannot* replace server-side validation because client-side code can be bypassed by disabling JavaScript or crafting direct HTTP API calls via tools like Postman or cURL. Defense-in-depth requires validation on both layers.

Q2: How does the application avoid intimidating users with premature error messages?

Answer: We implemented a two-fold state strategy using `errors` and `touched` dictionaries. Error messages are only rendered if touched[fieldName] === true. This ensures that pristine fields do not display glaring red errors before the user has even clicked or typed in them.

Q3: What role does useMemo play in the pricing calculation?

Answer: useMemo memoizes the calculated subtotal, workshop fees, and applied discount, only re-evaluating when its dependencies (passTier, selectedWorkshops, promoCode) change. This prevents expensive recalculations on unrelated state changes such as typing in text inputs.

8. Conclusion & Academic Declaration

The web application **EventSphere 2026** successfully fulfills all requirements set forth in the Web Technology laboratory assignment. It demonstrates client-side form validation across diverse data types, provides meaningful error and success notifications, features an interactive and responsive user interface, has been thoroughly verified with 42 automated test cases, and is accessible live on GitHub Pages.

Student Signature: _____ **Date of Submission:** October 31, 2026

Evaluator / Faculty Signature:
_____ **Marks / Grade Awarded:** _____ / _____